

# Rate Limiting

**Rate Limiting** is a technique used to control how many requests a user, client, IP, or service can make within a given time period.

It protects systems from overload, abuse, and unfair resource usage.

## 1. Core Idea

Limit incoming traffic by defining rules such as:

None

100 requests / minute per user

10 login attempts / hour per IP

1000 API calls / second globally

If limit is exceeded:

- Reject request (429 Too Many Requests)
- Delay request
- Queue request
- Apply degraded response

## 2. Why Rate Limiting is Important

### ✓ Prevent System Overload

Protect backend servers, databases, and APIs from traffic spikes.

### ✓ Stop Abuse / Attacks

Helps defend against:

- DDoS attempts
- Brute-force login attacks
- API scraping
- Spam bots

### ✓ Fair Usage

Prevents one user from consuming all shared resources.

### ✓ Cost Control

Reduces excessive compute, bandwidth, and third-party API costs.

### 3. Where Rate Limiting is Applied

#### API Gateway

Most common place.

None

Client → API Gateway → Rate Limiter → Services

## Web Server / Reverse Proxy

Examples:

- NGINX
- Envoy

## Application Layer

Per-user business rules.

## CDN / Edge Layer

Throttle before traffic reaches origin.

# 4. Rate Limiting Strategies

### ✓ By User ID

None

100 requests/minute per user

## ✓ By IP Address

Useful for anonymous traffic.

## ✓ By API Key

Used in SaaS APIs.

## ✓ By Endpoint

Different limits for different APIs.

None

POST /payment → strict limit

GET /profile → higher limit

## ✓ Global Limit

Protect total infrastructure capacity.

# 5. Common Algorithms

## Token Bucket

Allows bursts while enforcing average rate.

None

Best for APIs with burst traffic

## Leaky Bucket

Smooths traffic into constant outflow.

## Fixed Window Counter

Simple counting per time window.

None

100 req/min resets every minute

## Sliding Window Log / Counter

More accurate and smoother than fixed window.

## 6. Example Flow

None

Limit = 5 req/min

User sends 7 requests

First 5 allowed

Last 2 rejected with 429

## 7. Response When Limit Exceeded

Use HTTP:

None

429 Too Many Requests

Retry-After: 60

Optional response body:

- Limit exceeded
- Retry after X seconds

## 8. Distributed System Challenges

### Multiple Servers

Need shared counters across instances.

Solutions:

- Redis centralized counters
- Sharded counters
- Consistent hashing

### Clock Synchronization

Important for time-window algorithms.

### High Performance

Limiter itself must be fast and scalable.

## 9. Best Practices

- Different limits for different endpoints
- Premium users get higher quotas



- Use soft + hard limits
- Log throttled requests
- Add CAPTCHA for abuse patterns
- Return clear retry info

## 10. Trade-Offs

| Approach       | Advantage      | Drawback               |
|----------------|----------------|------------------------|
| Fixed Window   | Simple         | Boundary burst problem |
| Sliding Window | Accurate       | More complex           |
| Token Bucket   | Burst-friendly | Parameter tuning       |
| Leaky Bucket   | Smooth traffic | Less burst tolerance   |

# 11. Real-World Use Cases

- Login protection
- Payment APIs
- SMS OTP sending
- Search APIs
- Social media posting limits
- Cloud platform quotas

## Mental Model

None

Rate limiting = Fairness + Protection + Stability

## One-Line Summary

Rate limiting controls request volume over time to protect systems, ensure fair usage, and maintain reliability using algorithms like token bucket, sliding window, and fixed window counters.